Sparse Ranking Models for the NLP Search Engine

A formal treatment of TF−IDF and Okapi BM25

Hamza Abdul Karim · F23607046 — May 2026

ABSTRACT

We document the two interchangeable ranking functions implemented in the engine. Both are unsupervised, depend only on corpus statistics, and produce strong baselines without any training data. We give the formal definitions used by the implementation, discuss the role of each parameter, and offer guidance on when to prefer one model over the other. Each formula is reproduced below as it is computed by the code.

1. Notation

Throughout this document we use the following symbols. Let D denote the indexed corpus and N=|D| its size. For a term t and a document d, tf(t,d) is the number of occurrences of t in d; df(t) is the number of documents containing t; |d| is the document length in tokens after preprocessing; and avgdl is the average document length over the corpus.

2. TF−IDF and Cosine Similarity

Each document is represented as a sparse vector indexed by terms. The weight of term t in document d uses logarithmic term-frequency weighting:

$$\text{tf\_weight}(t,d) = 1 + \log(\text{tf}(t,d)) \quad \text{if } \text{tf}(t,d) > 0 \tag{2.1}$$

and a smoothed inverse document frequency:

$$\text{idf}(t) = \log\left(\frac{N+1}{\text{df}(t)+1}\right) + 1 \tag{2.2}$$

The composite weight is the product of the two:

$$w(t,d) = \text{tf\_weight}(t,d) \cdot \text{idf}(t) \tag{2.3}$$

A query q is mapped into the same vector space using Eq. (2.3). Documents are ranked by cosine similarity:

$$\cos(q,d) = \frac{\sum_t q[t]\,d[t]}{\sqrt{\sum_t q[t]^2} \cdot \sqrt{\sum_t d[t]^2}} \tag{2.4}$$

Cosine values lie in [0, 1] and are insensitive to absolute term counts, which makes TF−IDF a predictable baseline that is easy to debug. The implementation in `src/tfidf.py` caches every document vector at construction time so each query only recomputes the query vector and a sparse dot product.

3. Okapi BM25

BM25 fixes two empirical weaknesses of plain TF−IDF: it saturates the contribution of repeated terms, and it normalizes for document length. The score for a query–document pair is

$$\mathrm{BM25}(q, d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{\mathrm{tf}(t, d)\,(k_1 + 1)}{\mathrm{tf}(t, d) + k_1 \left(1 - b + b\,\frac{|d|}{\mathrm{avgdl}}\right)}$$

(3.1)

with the Robertson–Spärck Jones smoothed IDF

$$\mathrm{IDF}(t) = \log\left(\frac{N - \mathrm{df}(t) + 0.5}{\mathrm{df}(t) + 0.5} + 1\right)$$

(3.2)

3.1 Parameters

Parameter	Default	Effect
k1	1.5	Term-frequency saturation. Smaller values saturate faster.
b	0.75	Length normalization. b=0 ignores |d|; b=1 fully penalizes long docs.

4. Damerau–Levenshtein Suggestions

When a query term is not present in the index vocabulary, the engine suggests near-matches using Damerau–Levenshtein distance — the minimum number of insertions, deletions, substitutions, and transpositions to transform one string into another. The standard recurrence used by `src/spell_check.py` is

$$d_{i,j} = \min\{\, d_{i-1,j} + 1,\ d_{i,j-1} + 1,\ d_{i-1,j-1} + \mathbf{1}_{a_i \neq b_j},\ d_{i-2,j-2} + 1 \,\}$$

(4.1)

The transposition term (the last alternative inside the brackets) applies only when the last two characters of the prefixes are swapped, i.e. $a_i = b_{j-1}$ and $a_{i-1} = b_j$.

5. Evaluation Metrics

The engine ships with the standard ranked-retrieval metrics in `src/metrics.py`. Let R_q be the set of documents relevant to a query q, let $L_q(k)$ be the top-k ranked documents returned for q, and let $r_i = 1$ if the document at rank i is relevant and 0 otherwise. We define

$$P@k = \frac{|L_q(k) \cap R_q|}{k}, \qquad R@k = \frac{|L_q(k) \cap R_q|}{|R_q|}$$

(5.1)

and the average precision over a single query

$$\mathrm{AP}(q) = \frac{1}{|R_q|} \sum_{i=1}^{|L_q|} r_i \, P@i$$

(5.2)

The mean average precision (MAP) is the arithmetic mean of AP over a query set. The mean reciprocal rank uses only the rank of the first relevant document:

$$\mathrm{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\mathrm{rank}_q}$$

(5.3)

Finally, normalized discounted cumulative gain uses graded relevance scores rel_i:

$$\mathrm{nDCG@}k = \frac{\sum_{i=1}^{k} (2^{rel_i} - 1) / \log_2(i+1)}{\sum_{i=1}^{k} (2^{rel_i^*} - 1) / \log_2(i+1)}$$

(5.4)

where the denominator is the ideal DCG, computed by sorting the relevance grades in decreasing order before applying the logarithmic position discount.

6. Choosing a Model

- TF−IDF — predictable, deterministic, and scores in [0, 1]; preferred for small corpora and ensembles where a normalised signal is required.
- Okapi BM25 — the general-purpose default. Outperforms TF−IDF on virtually every benchmark and handles long documents gracefully through length normalization.